

IE3042 – SSS ASSIGNMENT



IT23545212

DENCIL P. A.N

Contents

1. <u>Task 1: Game Selection and Environment Setup</u>	3
1.1 Game Justification	3
1.2 Environment and Tools Setup	3
2. Task 2: Static and Dynamic Analysis	5
2.1 Static and Dynamic Disassembly	5
2.2 Register and Memory Identification	5
3. Task 3: Assembly-Level Modification	6
3.1 Instruction Identification	6
3.2 Modification Implementation (NOPing)	6
3.3 Permanent Binary Patching	6
4. Task 4: Security Analysis and Mitigation	8
4.1 Vulnerability Discussion	8
4.2 Proposed Modern Mitigation Strategies	8
5. Conclusion	9

1. Task 1: Game Selection and Environment Setup

1.1 Game Justification

For this project, **Cave Story (NXEngine-Evo v2.6.5-1)** was selected. Although the engine is a 64-bit refactor, it preserves the legacy logic of the original 2004 release. This selection is highly relevant to security because:

- **Lack of Modern Protections:** It lacks basic anti-debugging and integrity checks common in modern software.
- **Deterministic Memory:** It provides a clear demonstration of how user-mode applications manage critical state variables (Health) in unprotected RAM.
- **Legacy Logic:** It utilizes standard arithmetic assembly instructions (DEC, SUB) for state changes, providing an ideal attack surface for instruction patching.

1.2 Environment and Tools Setup

The analysis was conducted in a **Windows 11 Sandboxed Environment** to isolate the process. The primary tool utilized was **x64dbg**, an open-source binary debugger. x64dbg was chosen for its robust **TitanEngine** backend, which allows for real-time register tracking and the ability to apply permanent patches to the Portable Executable (PE) file.

Screenshot before :



2. Task 2: Static and Dynamic Analysis

2.1 Static and Dynamic Disassembly

Upon loading nx.exe into x64dbg, the entry point was identified. By performing a **Constant Search** for the player's initial health value (3 HP), the initialization function was located.

- **Key Function identified:** The health management routine responsible for collision-based state changes.

2.2 Register and Memory Identification

Through systematic debugging, the following was identified:

- **Base Pointer:** The **RCX register** acts as a pointer to the Player Object.
- **Memory Address:** The health variable is stored at a relative offset: $[rcx + 01]$.
- **Runtime Behavior:** When a collision occurs, the game fetches the value from this address, passes it to the CPU for decrementing, and writes it back.

3. Task 3: Assembly-Level Modification

3.1 Instruction Identification

The exact instruction responsible for health reduction was isolated at address **00007FF7DC72EF78**:

- **Original Instruction:** dec dword ptr ds:[rcx]
- **Opcode:** FF 09
- **Technical Justification:** This instruction directly decrements the 4-byte value at the address stored in RCX.

3.2 Modification Implementation (NOPing)

To neutralize this logic, a **NOP (No-Operation) Patch** was applied.

- **Action:** The DEC instruction was replaced with 90 90.
- **Before Modification:** Player health reduced from 3 to 2 upon enemy contact.
- **After Modification:** The CPU ignores the decrement logic. Health remains static at 3 HP regardless of collisions.

3.3 Permanent Binary Patching

Using the **Patches (Ctrl+P)** engine in x64dbg, the memory-resident changes were committed to a new executable: nx_modified.exe. This demonstrates the transition from a dynamic exploit to a permanent software modification.

Screenshot memory allocation and patching:

nx.exe - PID: 26680 - Module: nx.exe - Thread: Main Thread 6996 - x64dbg

File View Debug Tracing Plugins Favourites Options Help Aug 19 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbol

00007FFDC72EF2E 33FF xor edi,edi
 00007FFDC72EF2E 0FB641 01 movzx eax,byte ptr ds:
 48:8859 20 mov rbx,qword ptr ds:
 84C0 test al,al
 74 4E je nx.7FFDC72EF8A
 0FB600 movzx edx,al
 83EA 01 sub edx,1
 72 2E je nx.7FFDC72EF7E
 83EA 01 sub edx,1
 74 1A je nx.7FFDC72EF63
 83FA 01 cmp edx,1
 75 46 jnc nx.7FFDC72EF94
 FF49 04 dec dword ptr ds:[rc
 8379 04 E5 cmp dword ptr ds:[rc
 7F 3D jg nx.7FFDC72EF94
 40:8879 01 mov byte ptr ds:[rcx
 8379 08 mov dword ptr ds:[rc
 8979 10 mov dword ptr ds:[rc
 EB 31 jmp nx.7FFDC72EF94
 FF41 10 inc dword ptr ds:
 8379 10 2A cmp dword ptr ds:
 7C 28 jl nx.7FFDC72EF94
 C641 01 10 mov byte ptr ds:[rc
 EB 22 jmp nx.7FFDC72EF94
 8371 10 01 xor dword ptr ds:[rc
 74 1C je nx.7FFDC72EF94
 FF49 04 dec dword ptr ds:
 8379 04 ED cmp dword ptr ds:[rc
 7F 13 jg nx.7FFDC72EF94
 C641 01 02 mov byte ptr ds:[rc
 8979 10 mov dword ptr ds:
 EB 04 jmp nx.7FFDC72EF94
 40:8839 cmp byte ptr ds:[rc
 74 05 je nx.7FFDC72EF94
 E8 0CFFFF call nx.7FFDC72EF94
 48:8BC8 mov rcx,rbx

Hide FPU
 RAX 0000000000000001
 RBX 0000000000000000
 RCX 00000141F8F6B384
 RDX 0000000000000000
 RBP 0000000020AEFFB30
 RSP 0000000020AEFF6A0
 RSI 0000000000000000
 RDI 0000000000000000
 R8 0000000000000002
 R9 000000000000002A
 R10 0000000000000002
 R11 000000000000002D
 R12 00000141F10438A0
 R13 00000000000025C770

NXEngine-evo 2.6.5

00007FFDC72EF7E 33FF xor edi,edi
 00007FFDC72EF7E 0FB641 01 movzx eax,byte ptr ds:
 48:8859 20 mov rbx,qword ptr ds:
 84C0 test al,al
 74 4E je nx.7FFDC72EF8A
 0FB600 movzx edx,al
 83EA 01 sub edx,1
 72 2E je nx.7FFDC72EF7E
 83EA 01 sub edx,1
 74 1A je nx.7FFDC72EF63
 83FA 01 cmp edx,1
 75 46 jnc nx.7FFDC72EF94
 FF49 04 dec dword ptr ds:[rc
 8379 04 E5 cmp dword ptr ds:[rc
 7F 3D jg nx.7FFDC72EF94
 40:8879 01 mov byte ptr ds:[rcx
 8379 08 mov dword ptr ds:[rc
 8979 10 mov dword ptr ds:[rc
 EB 31 jmp nx.7FFDC72EF94
 FF41 10 inc dword ptr ds:
 8379 10 2A cmp dword ptr ds:
 7C 28 jl nx.7FFDC72EF94
 C641 01 10 mov byte ptr ds:[rc
 EB 22 jmp nx.7FFDC72EF94
 8371 10 01 xor dword ptr ds:[rc
 74 1C je nx.7FFDC72EF94
 FF49 04 dec dword ptr ds:
 8379 04 ED cmp dword ptr ds:[rc
 7F 13 jg nx.7FFDC72EF94
 C641 01 02 mov byte ptr ds:[rc
 8979 10 mov dword ptr ds:
 EB 04 jmp nx.7FFDC72EF94
 40:8839 cmp byte ptr ds:[rc
 74 05 je nx.7FFDC72EF94
 E8 0CFFFF call nx.7FFDC72EF94
 48:8BC8 mov rcx,rbx

dword ptr ds:[rcx+04]=[00000141F8F6B384]=FFFFFFFF

.text:00007FFDC72EF78 nx.exe:54EF78 #4E378

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex
00007FFB9A380000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00
00007FFB9A380010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
00007FFB9A380020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00007FFB9A380030	00 00 00 00 00 00 00 00 00 00 00 00 E0 00 00 00
00007FFB9A380040	0E 1F BA 0E 00 84 09 CD 21 88 01 4C CD 21 54 68
00007FFB9A380050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F
00007FFB9A380060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20
00007FFB9A380070	6D 6F 64 65 2E 00 00 0A 24 00 00 00 00 00 00 00
00007FFB9A380080	F8 B1 A4 00 BC D0 CA 03 BC D0 CA 03 BC D0 CA 03
00007FFB9A380090	BC D0 CA 03 BE D0 CA 03 C5 51 CE C2 D0 CA 03
00007FFB9A3800A0	C5 51 CA 02 BD D0 CA 03 C5 51 C9 02 8D D0 CA 03

Command: Commands are comma separated (like assembly instructions): mov

Paused INT3 breakpoint at nx.00007FFDC72EF78!

Time Wasted Debugging: 0:06:38:48

nx.exe - PID: 16572 - Module: sd2_mixer.dll - Thread: 31064 - x64dbg

File View Debug Tracing Plugins Favourites Options Help Aug 19 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Symbols Source References Threads Handles Trace

000000000078A044 0200 add al,byte ptr ds:[rax]
 000000000078A045 0000 add byte ptr ds:[rax],al
 000000000078A046 0000 add byte ptr ds:[rax],al
 000000000078A047 0000 add byte ptr ds:[rax],al
 000000000078A048 0000 add byte ptr ds:[rax],al
 000000000078A049 0000 add byte ptr ds:[rax],al
 000000000078A04A 3A02 cmp al,byte ptr ds:[rax]
 000000000078A04B 0000 add byte ptr ds:[rax],al
 000000000078A04C 0100 add dword ptr ds:[rax],eax
 53 push rbx
 007400 72 add byte ptr ds:[rax+eax*72],dh
 000000000078A04E 0000 add byte ptr ds:[rcx],ch
 6E OUTEB
 000000000078A04F 0000 add byte ptr ds:[rcx],ah
 4E:0069 00 add byte ptr ds:[rcx],r13b
 000000000078A050 0000 MOVSB
 000000000078A051 0000 add byte ptr ss:[rbp],bp
 43:006E 00 add byte ptr ds:[r14],bp
 4E:006F 00 add byte ptr ds:[rdi],ch
 0000 add byte ptr ds:[rax],al
 10 MOVSB
 000000000078A053 0200 add al,byte ptr ds:[rax]
 0000 add byte ptr ds:[rcx],al
 000000000078A055 0030 add byte ptr ds:[rax],dh
 000000000078A056 0030 add byte ptr ds:[rax],dh
 000000000078A057 3900 xor byte ptr ds:[rax],eax
 000000000078A058 3900 xor byte ptr ds:[rax],al
 34 00 MOVSB
 000000000078A05A 62 add byte ptr ds:[rax],dh
 0000 add byte ptr ds:[rcx],al
 000000000078A05C 0030 add byte ptr ds:[rax],ah
 0000 add byte ptr ds:[rcx],al
 000000000078A05E 0001 add byte ptr ds:[rcx],al
 000000000078A05F 0001 add byte ptr ds:[rcx],al
 000000000078A060 0001 add byte ptr ds:[rcx],al

Hide FPU
 RAX 00000000000001E6 L'G'
 RBX 0000000000000000
 RCX 0000000000000018
 RDX 0000000000000040
 RBP 0000000000000000
 RSP 0000000000000000
 RSI 00007FFB9A3F4D60 ntdll.00007FFB9A3F4D60
 RDI 000001D5E551A80
 R8 0000000000000010
 R9 0000000000000008
 R10 0000000000000018
 R11 0000000000000018 ntdll.00007FFB9A4A410
 R12 0000000000000000
 R13 0000000000000000
 R14 00000000000000FF ntdll.00007FFB9A3F2AD0
 R15 00000000000000FF ntdll.00007FFB9A4E5714
 RIP 00007FFB9A4E5714 ntdll.00007FFB9A4E5714
 RFLAGS 0000000000000024
 CR0 0 0 0 0 0 0 0
 CR2 0 0 0 0 0 0 0
 CR3 0 0 0 0 0 0 0
 CR4 0 0 0 0 0 0 0
 CR8 0 0 0 0 0 0 0
 LastError 00000000 (ERROR_SUCCESS)

Default (x64 FPU)
 1: rcx 0000000000000018 0000000000000018
 2: rcx 000001D5E551A80 000001D5E551A80
 3: rcx 0000000000000000 0000000000000000
 4: r9 0000000000000000 0000000000000000
 5: [rsp+28] 0000000000000000 0000000000000000

al=E6 'a'
 byte ptr ds:[rax]=1E6 'L'G'=???
 .rsrc:000000000078A044 sd2_mixer.dll:5260A4 #IDEA4 <1f6_value>

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch Locals Struct

Address	Hex	ASCII
00007FFB9A380000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	YV.....
00007FFB9A380010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	@.....
00007FFB9A380020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00007FFB9A380030	00 00 00 00 00 00 00 00 00 00 00 00 E0 00 00 00	@.....
00007FFB9A380040	0E 1F BA 0E 00 84 09 CD 21 88 01 4C CD 21 54 68	[.LITh
00007FFB9A380050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	carn
00007FFB9A380060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS
00007FFB9A380070	6D 6F 64 65 2E 00 00 0A 24 00 00 00 00 00 00 00	mode:.....
00007FFB9A380080	B1 A4 50 BC D0 CA 03 BE D0 CA 03 C5 51 CE C2 D0 CA 03	esp=0E..N6..N6..
00007FFB9A380090	BC D0 CA 03 BE D0 CA 03 C5 51 CE C2 D0 CA 03	N6..N6..AQ..N6..
00007FFB9A3800A0	C5 51 CA 02 BD D0 CA 03 C5 51 C9 02 8D D0 CA 03	AD..N6..AD..BE..
00007FFB9A3800B0	C5 51 C2 02 15 D0 CA 03 C5 51 CF 01 A2 D0 CA 01	AD..N6..AD..S6..
00007FFB9A3800C0	C5 51 35 01 8D D0 CA 03 C5 51 CA 02 8D D0 CA 01	AD..N6..N6..N6..

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

nx.exe: 00007FFDC72EF78 -> 00007FFDC72EF78 (0x00000000 bytes)

Time Wasted Debugging: 0:06:10:04

4. Task 4: Security Analysis and Mitigation

4.1 Vulnerability Discussion

This attack is possible due to several fundamental security flaws in legacy software architecture:

1. **Trusting the Client:** The game assumes that the local memory environment is "clean" and does not verify the integrity of its own instructions during runtime.
2. **Lack of Anti-Debugging:** The application does not call `IsDebuggerPresent()` or utilize `NtQueryInformationProcess`, allowing an attacker to hook the process and manipulate the EIP (Instruction Pointer).
3. **No Code Signing:** The OS executes the patched `nx_modified.exe` without verifying a digital signature, allowing unauthorized versions of the software to run.

4.2 Proposed Modern Mitigation Strategies

To prevent such manipulations in modern systems, I propose the following:

- **Anti-Debugging Traps:** Implement guard strips in the code that detect the presence of a debugger and terminate the process.
- **Checksum Verification (Self-Checksumming):** The application should perform a hash check of its .text section during startup. If the hash differs (e.g., due to NOPs), the program should refuse to load.
- **Address Space Layout Randomization (ASLR):** While present in modern OSs, making the health pointer dynamic or "obfuscated" via XOR operations would make memory scanning significantly harder.
- **Memory Protection:** Using **Data Execution Prevention (DEP)** and read-only memory segments for critical constants.

5. Conclusion

The project successfully demonstrated the reverse engineering lifecycle—from environment setup to the deployment of a modified binary. The vulnerability found highlights the critical need for **Integrity Checking** and **Anti-Tamper** mechanisms in modern software development.

Screenshot after :



VIDEO LINK : IT23545212_SSS_Assignment.zip